

Cortex: A Fixed-Point Theory of Governed Coding Agents

Conditional semantics and an evaluation methodology for governed coding agents

Marius-Constantin Dinu Florian Zeba

Alpha Omega Labs

September 14, 2026

Canonical paper and latest results:

<https://benchmark.cortex.a2olabs.com/docs.html>

<https://benchmark.cortex.a2olabs.com/results.html>

Benchmark source: <https://github.com/Xpitfire/cortex-gauntlet>

Abstract

A *coding agent* combines a large language model (LLM) with a control loop, or *harness*, that can plan, edit, and execute code. We study **Cortex**, a supervisory layer coupling pre-execution governance, requirement tracking, and iterative validation and repair. We give a **conditional** finite-lattice model of requirement closure: for a fixed finite requirement set and a sound, monotone validation rule with persistent evidence, exhaustive iteration reaches the least fixed point after at most $|R|$ strict increases. Fair consequence scheduling reaches the same closure. These are properties of the specified operator, **not an unconditional completion or safety guarantee for the stochastic implementation**. Code repairs can invalidate previous evidence, validators can miss defects, and a budgeted run can stop before closure.

We define the benchmark’s bounded trajectory-similarity and build-gated capability scores, attack-success estimators, Wilson score intervals, and case-level bootstrap summaries. A separate probabilistic model gives an expected completion-time bound only under a uniform progress assumption. Effective-feedback accounting counts newly certified requirements; it is not a decision-theoretic value-of-information estimate. The five benchmark families provide a framework for raw-versus-governed comparisons. Historical summaries and schematic figures do not by themselves establish model-matched causal gains, theorem conformance, or long-horizon scaling; Section 7 separates their evidence status from the formal results.

Contributions

- A typed action-filter model and a conditional finite-lattice model of requirement closure (Sections 3–4).
- A proof of finite closure, oracle-relative soundness, and fair-schedule independence under explicit assumptions (Theorem 1), plus accounting and conditional hitting-time bounds (Propositions 3–4).
- Definitions and domain restrictions for the implemented evaluation metrics (Section 5).
- A five-family evaluation protocol, with implementation and historical-evidence limitations made explicit rather than treated as experimental conclusions (Sections 6–9).

1. Introduction

An LLM coding harness manages context and turns model output into executed actions — shell commands, file edits, tool calls. Capability and *control* are distinct axes: a highly capable harness can still be talked into an unsafe action [46, 20], and a safe but myopic one can drop requirements partway through a long task [21]. **Cortex** is a *meta-level control layer* that supplies what the base harness lacks on each axis. For safety it adds **governance**: deterministic pre-execution checks and capability/dependency policies that constrain which actions may run. For capability it adds **orchestration**: instruction analysis and long-horizon planning that structure the task. Tying the two together is an iterative validate–repair loop — the **Synapse loop** — that attempts verified completion within a budget. Cortex does not replace the base model; it *supervises* it.

The distinction matters. Governance *alone* — deterministic filtering of individual actions — can neither plan a long-horizon task nor judge whether a language-level request is safe to act on; a meta-level controller both constrains *and* directs, and its validation may be semantic (an LLM-checked requirement or contract) as well as a fixed rule. The empirical comparisons in this report come from a reproducible benchmark we refer to as **Gauntlet**.

Test-time scaling improves agents by spending more inference compute — repeated sampling, search, revision [26, 27] — extending the training scaling laws [24, 25] to inference. The effective-feedback perspective [21] motivates tracking informative, valid, non-redundant, retained feedback rather than only tokens or tool calls. Our requirement count is one operational proxy for retained progress, not a reproduction of a scaling law. Classical value of information [22] and value of computation [23] additionally require beliefs, utilities, and computation costs; a ledger update alone does not determine either value.

We address (i) *what an idealized supervisory layer guarantees under stated assumptions* and (ii) *how its implemented behavior can be evaluated*. The least fixed point in (i) is the closure of a specified validation rule, which may leave required work unresolved. The metrics and comparison protocol in (ii) do not establish that a particular implementation satisfies the theorem’s hypotheses or that historical arm differences isolate governance alone.

2. Related Work and Theoretical Lineage

Neurosymbolic and cognitive computation. A neural generator constrained and verified by a symbolic layer is the defining shape of the “third wave” of neurosymbolic AI [6, 7]. The completion loop is, structurally, the recognize–act cycle of classical cognitive architectures — the problem-space hypothesis of Newell & Simon [8], SOAR [9], and ACT-R [10] — in which rules fire repeatedly until a goal or impasse. We make that cycle precise as a fixed-point computation.

Fixed-point semantics. Our convergence argument rests on the least-fixed-point semantics of monotone operators: van Emden & Kowalski’s immediate-consequence operator and its least fixed point as the meaning of a logic program [2], the Knaster–Tarski lattice fixed-point theorem [3], and Kleene iteration [4]. Where an operator is a contraction on a metric space, Banach’s theorem [5] gives uniqueness and a rate; we use the lattice formulation because the requirement state space is naturally a finite complete lattice.

Evaluation. Capability over trajectories is scored by VERTEX, defined in Section 5.1: embedding cross-similarity against a reference descriptor set, with a presence component (BERTScore-style bidirectional matching [11]) and an order component (distance-decayed dynamic time warping [12]),

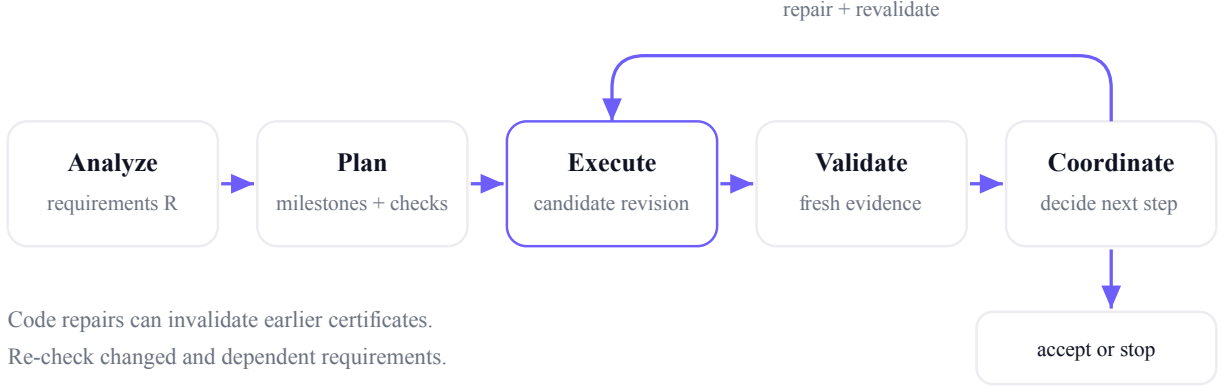


Figure 1: Schematic execution-validation-repair architecture. Only under A3 does an exhaustive pass represent the deterministic consequence operator. Real code repairs can invalidate earlier certificates; a no-progress stochastic attempt is not fixed-point detection.

each affinely recalibrated against the matrix’s mean-similarity baseline. Trajectory-level scoring of agents by embedding cross-similarity is an established line of work [1, 19]: capability and quality are read from the trajectory, not only from the final output. Code capability uses the unbiased pass@ k estimator [15]; success rates carry Wilson score intervals [13] and bootstrap intervals [14]. Qualitative judging follows the LLM-as-judge literature — G-Eval [16], MT-Bench [17], Prometheus [18] — whose bias analyses motivate, but do not establish, our judge validity; safety draws on the jailbreak [46] and real-world prompt-injection [20] literatures.

Scaling, test-time compute, and effective feedback. Training scaling laws relate compute, data, and loss [24, 25]; inference-time search and revision trade compute for accuracy [26, 27]. Effective-feedback work [21] motivates measuring usable feedback. We count new requirement certificates as $\text{efc}(S) = |\Phi(S) \setminus S|$. This finite-set accounting identity neither derives an empirical scaling law nor establishes that equal counts represent equal information or utility.

Iterative feedback, verification, and metareasoning. Closed-loop harnesses that reason, act, and revise — ReAct [31], Self-Refine [29], Reflexion [30] — and step-level verification / process rewards [28] are heuristic instances of execute-validate-repair, typically evaluated on repository tasks such as SWE-bench [32]. Our theorem applies to any operator satisfying its assumptions, not uniquely to Cortex; casting an implementation as a monotone operator is a proof obligation, not a guarantee obtained by naming its loop. Value of information [22] and rational metareasoning [23] provide decision-theoretic context, but no utility model or optimal computation-selection policy is proved here.

3. Preliminaries and Notation

We write $\mathbb{N}_0 = \{0, 1, \dots\}$, $\mathbb{N}_+ = \{1, 2, \dots\}$, $\mathbb{R}$ for the reals, and $\mathbb{I} = [0, 1]$. For finite X , $|X|$ denotes cardinality and 2^X the power set. For nonempty finite or countable X , $\Delta(X) = \{p \in \mathbb{R}_{\geq 0}^X : \sum_{x \in X} p_x = 1\}$ is the set of discrete probability distributions. For $d \in \mathbb{N}_+$, $\mathbb{S}^{d-1} = \{v \in \mathbb{R}^d : \|v\|_2 = 1\}$, with Euclidean inner product $\langle u, v \rangle \in [-1, 1]$. For real $a \leq b$, $\text{clamp}(x; a, b) = \min(\max(x, a), b)$.

Order-theoretic objects. A complete lattice $(L, \sqsubseteq)$ has a least upper bound and a greatest lower

bound for every subset, including bottom and top elements. A map $f : L \rightarrow L$ is *monotone* when $x \sqsubseteq y$ implies $f(x) \sqsubseteq f(y)$, and *inflationary* when $x \sqsubseteq f(x)$. These are distinct properties. A fixed point satisfies $f(x) = x$. A monotone map on a complete lattice has a least fixed point $\text{lfp}(f)$ [3].

Agent objects. Let $\mathcal{A}$ and $\mathcal{O}$ be nonempty countable action and observation spaces, and $\mathcal{H}$ the finite histories of action–observation pairs. A base policy is $\pi : \mathcal{H} \rightarrow \Delta(\mathcal{A})$; $H \in \mathcal{H}$ denotes a history. A hook $h_j(H, a) \in \{0, 1\}$ blocks when it returns 1. A distinguished outcome $\perp_a \notin \mathcal{A}$ denotes refusal without execution, not the lattice bottom.

Task objects. Fix a finite requirement set $R = \{r_1, \dots, r_N\}$, $N \in \mathbb{N}_0$, and required subset $R_{\text{req}} \subseteq R$. The abstract state $S \subseteq R$ records certified requirements. The lattice $(2^R, \subseteq)$ has join $\cup$, meet $\cap$, bottom $\emptyset$, and top R . The abstraction $\nu : 2^R \times R \rightarrow \{0, 1\}$ determines which further certificates can be derived from S . Representing evidence by requirement identifiers alone requires the sufficiency assumption in Section 4.2; an actual workspace, tests, and execution history contain more information.

Metric objects. Let $\mathcal{D}$ be finite text descriptors and $\mathcal{D}^* = \bigcup_{m \in \mathbb{N}_0} \mathcal{D}^m$ their finite sequences. The embedding φ returns a unit vector for nonzero embeddings; the implementation uses the zero vector for a descriptor with no embedding features. Thus its full codomain is $\mathbb{S}^{d-1} \cup \{0\}$, and an exact text match need not score 1 for a zero vector. Signals $s_k \in \mathbb{I}$ combine with weights $w \in \Delta(\{1, \dots, K\})$, $K \in \mathbb{N}_+$.

4. The Control Layer: Governance and Orchestration

The action filter and requirement-closure operator describe different contracts. The first restricts proposed actions at a specified history; the second describes an idealized validation closure. Neither alone proves real-world task correctness.

4.1 Governance as a projection on the action space

At fixed H , let $\mathcal{A}_G(H) = \{a \in \mathcal{A} : h_j(H, a) = 0 \text{ for every } j\}$. On $\mathcal{A}_\perp = \mathcal{A} \cup \{\perp_a\}$ define

$$G_H(a) = \begin{cases} a, & a \in \mathcal{A}_G(H), \\ \perp_a, & a \notin \mathcal{A}_G(H). \end{cases}$$

In particular $G_H(\perp_a) = \perp_a$. Therefore $G_H \circ G_H = G_H$: it is an idempotent retraction onto $\mathcal{A}_G(H) \cup \{\perp_a\}$. The governed policy is the pushforward of the proposal policy:

$$\pi_G(a \mid H) = \pi(a \mid H) \mathbf{1}\{a \in \mathcal{A}_G(H)\} \quad (a \in \mathcal{A}), \quad \pi_G(\perp_a \mid H) = \sum_{a \notin \mathcal{A}_G(H)} \pi(a \mid H).$$

Blocked probability becomes refusal mass, **not renormalized admissible mass**. Conditional resampling until an admissible action is obtained would be a different policy and is undefined if the original policy assigns zero mass to admissible actions. Filtering leaves accepted actions unchanged at this history, but can change later behavior and task utility.

Assumption A1 (complete mediation and hook coverage). Every effect in the claimed protection boundary is checked before execution against authoritative context, without a check/use

race; every unsafe action in the specified threat class is blocked by at least one hook. Under A1 no action in that class executes. Pure predicates alone do not establish coverage, and neither false-positive rates nor semantic safety follow from idempotence. The benchmark must measure refusals and coverage separately.

4.2 Orchestration: the completion loop as a consequence operator

For a **fixed** oracle and requirement set define

$$\Phi(S) = S \cup \{r \in R : \nu(S, r) = 1\}.$$

Assumption A3 (oracle and evidence). Requirement identifiers are a sufficient summary for this deterministic rule. Validation is monotone: $S \subseteq T, \nu(S, r) = 1 \Rightarrow \nu(T, r) = 1$. For soundness, certificates derived from valid evidence are correct for the specified task, and previously valid certificates remain valid after every accepted transition. The union makes Φ inflationary; the additional monotonicity assumption makes it monotone. Append-only storage alone does **not** imply that assumption. For example, $R = \{a, b\}$ and $\nu(S, b) = \mathbf{1}\{a \notin S\}$ give $\Phi(\emptyset) = \{b\}$ but $\Phi(\{a\}) = \{a\}$.

Theorem 1 (finite closure, conditional soundness, fair-schedule independence).

Let $\Phi : 2^R \rightarrow 2^R$ be monotone and inflationary. Starting at $S_0 = \emptyset$, the iterates $S_{t+1} = \Phi(S_t)$ reach $S_* = \text{lfp}(\Phi)$ after at most $|R|$ strict increases. A further evaluation may be needed to detect stabilization. With A3's soundness and persistence conditions, all certificates in S_* remain valid. Define PASS at closure by $R_{\text{req}} \subseteq S_*$; PASS then certifies those requirements relative to the oracle, not all possible correctness properties. Starting at $\emptyset$, adding enabled consequences in any **fair** schedule reaches the same S_* .

Proof. Inflationarity gives an increasing chain. Each strict increase adds at least one of the $|R|$ elements, so the chain stabilizes. If F is any fixed point, then $S_0 \subseteq F$ and monotonicity implies $S_{t+1} = \Phi(S_t) \subseteq \Phi(F) = F$ by induction. The stabilized state is thus the least fixed point. Soundness follows by induction on certified additions, using both oracle soundness and persistence. For asynchronous addition, every added requirement is in $\Phi(S)$, so the same induction keeps the state below S_* . Fairness means no persistently enabled missing requirement is postponed forever. Since there can be only finitely many additions, the eventual state has no enabled missing requirement and is a fixed point; leastness makes it S_* . $\square$

The theorem proves derivability under ν , **not** that S_* is exactly the set of requirements achievable by arbitrary agents. Completeness of the oracle/generator would be an additional assumption. An unfair scheduler can omit enabled work forever. Detecting one unproductive *stochastic* attempt is not detecting a fixed point of a deterministic exhaustive consequence operator.

Implementation boundary. Real repair passes edit a shared artifact and can regress previously passing checks. A historical union of successful checks is not a current correctness certificate. The benchmark uses bounded retries, observation-dependent validation, and score-based candidate selection; these are not an implementation proof of A3 or exhaustive closure. Fresh validation of the final artifact is needed for PASS. Raw coding harnesses also take multiple internal actions; a single benchmark invocation is not a single consequence or a single requirement addition.

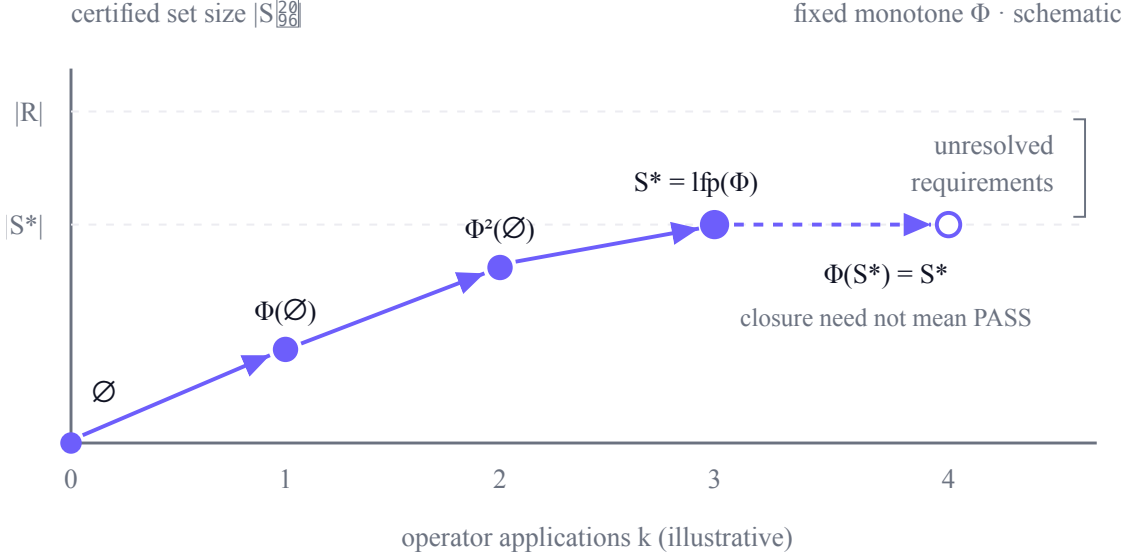


Figure 2: Illustrative Kleene ascent for the fixed monotone operator of Theorem 1, not a measured agent trace. At most $|R|$ strict increases occur; one further evaluation may detect closure. A fair schedule reaches the same least fixed point, which can leave required work unresolved.

4.3 Effective feedback and the value of computation

Define the count $\text{efc}(S) = |\Phi(S) \setminus S|$ for the abstract operator. It measures new certified requirements, motivated by retained-feedback accounts [21]. Under A3 these additions are valid, non-redundant, and retained. Their information content and utility are not necessarily equal.

Proposition 3 (effective-feedback accounting). If $S_0 = \emptyset$ and $S_T = S_* = \text{lfp}(\Phi)$ is the stabilized iterate, then $\sum_{t=0}^{T-1} |S_{t+1} \setminus S_t| = |S_*|$. If $R_{\text{req}} \subseteq S_*$, exactly $|R_{\text{req}}|$ of these additions are required certificates. The **total** number of additions before first completion can exceed $|R_{\text{req}}|$ because optional requirements can be added too.

Proof. The disjoint differences $S_{t+1} \setminus S_t$ partition S_* . Intersecting that partition with R_{req} gives the required-only identity. $\square$

One productive pass can add any number from 1 to $|R \setminus S|$. For example, $\Phi(S) = R$ completes arbitrarily many requirements in one pass. Consequently no lower bound on required passes, no impossibility for single-invocation harnesses, and no advantage over independent sampling follows from this accounting. An append-only ledger prevents duplicate **credit**, not duplicate computation. Positive decision-theoretic value of information or net value of computation requires an explicit utility/cost model [22, 23]; $\text{efc} > 0$ is not equivalent to either.

4.4 Probabilistic execution over the fixed-point lattice

A stochastic process is separate from the deterministic closure model. Let $\nu_{\text{obs}}(H, S, a, o, r) \in \{0, 1\}$ denote validation of an observed execution, and write $U(H, S, a, o) = S \cup \{r : \nu_{\text{obs}}(H, S, a, o, r) = 1\}$ for historical certificate accumulation. A distinct current-artifact validation set may shrink. For a

requirement-only, time-homogeneous Markov abstraction, **assume** the conditional law of the next requirement state depends on the past only through S . Then

$$P(S, S') = \sum_{\substack{a \in \mathcal{A}_\perp, o \in \mathcal{O} \\ U(S, a, o) = S'}} \pi_G(a \mid S) E(o \mid S, a), \quad \sum_{S' \subseteq R} P(S, S') = 1,$$

where $E(\cdot \mid S, a)$ is the observation distribution (including a refusal observation for $\perp_a$), and U, π_G, E admit the stated sufficient-state representation. For general history-dependent execution use $X_t = (H_t, S_t)$ instead, including all relevant environment state in the history. Aggregating its transitions by S alone does not in general produce a Markov chain. Historical certificate accumulation gives $P(S, S') = 0$ unless $S \subseteq S'$ but does not prove current-artifact correctness.

Completion as a hitting time. Let $C = \{S \subseteq R : R_{\text{req}} \subseteq S\}$ and $T_C = \inf\{t \in \mathbb{N}_0 : S_t \in C\}$, with $\inf \emptyset = \infty$. For an increasing process C is a closed set; make its states absorbing if the run stops at first completion. Specify the initial state S_0 for all probabilities and expectations.

Proposition 4 (conditional completion time). For a finite increasing Markov chain, suppose every reachable $S \notin C$ has $\Pr(S_{t+1} \supsetneq S_t \mid S_t = S) \geq \varepsilon > 0$. Then $\Pr(T_C < \infty) = 1$ and $\mathbb{E}[T_C] \leq (|R| - |S_0|)/\varepsilon$.

Proof. There are at most $|R| - |S_0|$ strict increases. Before completion the wait for each increase is dominated by a geometric variable of mean $1/\varepsilon$. Their sum bounds T_C and its expectation. A finite expectation implies almost-sure completion. $\square$

An incomplete absorbing state violates the progress assumption. Without that assumption, completion need not occur and $\mathbb{E}[T_C]$ may be infinite. Governance changes the execution law through the refusal-mass filter of Section 4.1; comparing unsafe-event probabilities additionally requires an explicit unsafe-event indicator on the augmented state, not merely a subset of satisfied requirements. A difference of expected completion times is meaningful only when both expectations are finite. No transition probabilities or completion-time estimates are supplied by the lattice theorem.

5. Evaluation

5.1 VERTEX trajectory similarity

This implementation of VERTEX combines descriptor cross-similarity with an order penalty, inspired by trajectory evaluation [1, 19]. For nonempty sequences $c = (c_1, \dots, c_m)$ and $r = (r_1, \dots, r_n)$, $m, n \in \mathbb{N}_+$, define $\mathbf{S}_{ij} = \langle \varphi(c_i), \varphi(r_j) \rangle \in [-1, 1]$. For unit embeddings this is cosine similarity; zero-vector handling is as in Section 3. Either empty sequence receives score 0 by convention. Figure 3 shows the two components.

Presence (unordered). A BERTScore-style bidirectional best match [11], where P (precision) measures how well each candidate descriptor is matched by some reference and Q (recall) how well each reference descriptor is matched by some candidate:

$$\begin{aligned}
P &= \frac{1}{m} \sum_{i=1}^m \max(0, \max_j \mathbf{S}_{ij}), \\
Q &= \frac{1}{n} \sum_{j=1}^n \max(0, \max_i \mathbf{S}_{ij}), \\
F &= \begin{cases} 2PQ/(P+Q), & P+Q > 0, \\ 0, & P+Q = 0. \end{cases}
\end{aligned}$$

Thus $P, Q, F \in \mathbb{I}$ and $\min(P, Q) \leq F \leq \max(P, Q)$. Negative similarities still enter the order cost and the matrix-mean baseline below.

Order. Dynamic time warping [12] minimizes over paths Γ from $(1, 1)$ to (m, n) with steps $(1, 0)$, $(0, 1)$, or $(1, 1)$. For finite $\lambda \geq 0$, the cost is $d(i, j) = (1 - \mathbf{S}_{ij})(1 + \lambda|(i-1)/m - (j-1)/n|)$:

$$\text{DTW} = \min_{\gamma \in \Gamma} \sum_{(i,j) \in \gamma} d(i, j), \quad D = \max\left(0, 1 - \frac{\text{DTW}}{m+n}\right) \in \mathbb{I},$$

The path has between $\max(m, n)$ and $m+n-1$ entries. The denominator $m+n$ is a chosen normalization, not the path length or an upper bound on the **cost**.

Matrix-mean recalibration. For $0 < \epsilon < 1$, let $b = \text{clamp}((mn)^{-1} \sum_{i,j} \mathbf{S}_{ij}; 0, 1 - \epsilon)$ and $\eta_b(x) = \text{clamp}((x-b)/(1-b); 0, 1)$. The raw mean lies in $[-1, 1]$, and the clamp ensures a positive denominator. It is the mean of this candidate-reference pair, **not** an independently estimated null expectation. For finite $\alpha \in [0, 1]$:

$$\text{VERTEX}(c, r) = \alpha \eta_b(F) + (1 - \alpha) \eta_b(D) \in \mathbb{I}.$$

The benchmark configuration fixes $\alpha = 0.6$, $\lambda = 0.5$, and $\epsilon = 0.01$ (the baseline-clamp margin). Proposition 1 records the range and the exact sense of the calibration.

Proposition 1 (range and anchors). VERTEX lies in $\mathbb{I}$. The normalizer is nondecreasing on $\mathbb{R}$ and strictly increasing on $[b, 1]$, with $\eta_b(b) = 0$ and $\eta_b(1) = 1$. Identical sequences of nonzero unit embeddings score 1 in exact arithmetic.

Proof. All floored best matches lie in $\mathbb{I}$; the harmonic mean lies between its arguments. Costs are nonnegative, hence the clamped D lies in $\mathbb{I}$. The normalizer has the stated endpoints and slope $1/(1-b) > 0$ on its unclamped interval. The mixture is convex. Identical unit-vector sequences have unit diagonal similarities, $P = Q = 1$, and a zero-cost diagonal path, so $F = D = 1$. $\square$

There is **no universal chance floor**. A constant zero similarity matrix with $m = n$ has $F = 0, D = 1/2$, hence score 0.2 at the default mixture. With $m = 1, n = 9$ and $\lambda = 0.5$, the same zero similarities give path cost 11, $D = 0$, and score 0. Lengths, order, and embedding geometry therefore affect unrelated-input scores. Maxima need not strictly exceed a mean, and the DTW term need not concentrate near $(1+b)/2$. Comparisons require fixed extraction and embedding protocols; they are similarity comparisons, not calibrated probabilities or correctness certificates.

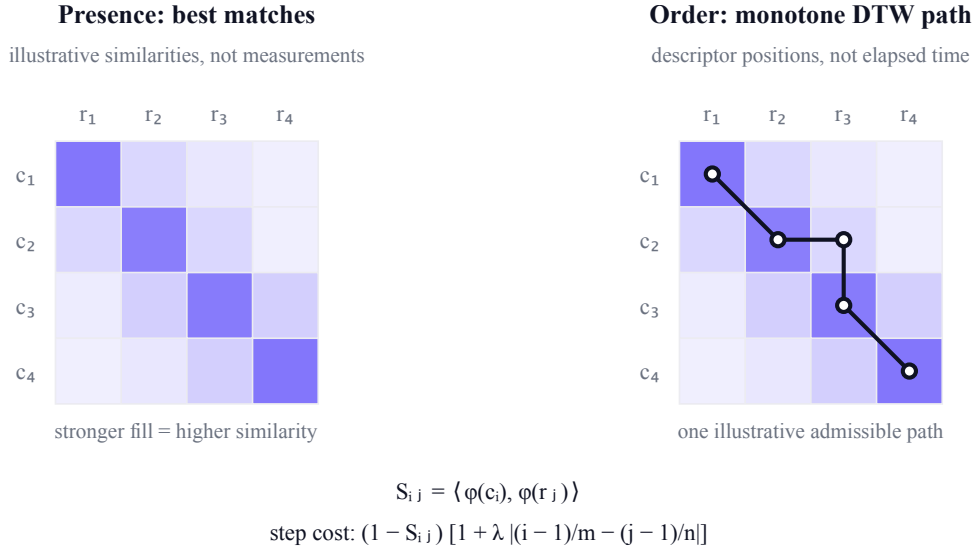


Figure 3: The two components of the VERTEX trajectory score (Section 5.1). Left: the cross-similarity matrix $S_{ij} = \langle \varphi(c_i), \varphi(r_j) \rangle$ between candidate descriptors c_i (rows) and reference descriptors r_j (columns); stronger color indicates higher similarity in this illustrative matrix. The *presence* component reads bidirectional best matches off this matrix (which reference behaviors appear, and how strongly). Right: the *order* component is a distance-decayed alignment path through the matrix whose step cost $(1 - S_{ij})(1 + \lambda |\frac{i-1}{m} - \frac{j-1}{n}|)$ penalizes matches that occur out of order. Both components are recalibrated against the mean-similarity baseline (Proposition 1).

5.1.1 VERTEX-QE: reference-free estimated references

VERTEX as defined in §5.1 requires a reference descriptor set r ; for the project task this is a hand-authored, hidden capability/architecture specification. Authoring one r per brief does not scale, and a single authored r commits the metric to one structure even though the brief is deliberately open about structure. VERTEX-QE removes the authored reference while **leaving the §5.1 kernel unchanged**: the presence component F (the bidirectional best-match score [11]), the order component D (the distance-decayed dynamic time warping [12]), and the chance normaliser η_b are computed exactly as before. **VERTEX-QE is reference-free, not order-free — it still computes the DTW term D** ; only the argument r changes, from an authored secret to an estimate $\hat{r}$ built from public sources.

The implementation estimates capability and architecture references **separately**. Write B for the public brief (distinct from baseline b), $\mathcal{C}$ for an architecture vocabulary, and c_i for candidate i . Capability descriptors are extracted from the brief’s enumerated outcomes. Architecture descriptors are deduplicated from $\mathcal{C}$ in a deterministic sequence. The shipped `arch_corpus.json` explicitly contains **synthetic canonical patterns**, not measurements extracted from curated repositories. A deduplicated union is not a distributional estimate of valid architectural alternatives; its sequence order also affects DTW although it does not describe execution time.

Let V_{cap} and V_{arch} be the two VERTEX scores, and $q_{\text{arch}} \in [0, 1]$ the resolver’s architecture agreement heuristic. The project signal is

$$V_P = \frac{0.7V_{\text{cap}} + 0.3q_{\text{arch}}V_{\text{arch}}}{0.7 + 0.3q_{\text{arch}}}.$$

Authored references use $q_{\text{arch}} = 1$. Agreement is not a calibrated probability of architectural correctness. COMET, SUPERT, and MAUVE [47, 48, 49] motivate reference estimation or distributional evaluation, but do not validate this rule.

Pool-level QE diagnostics additionally use leave-one-out peer descriptors supported by at least two peers. They are not inputs to the per-candidate composite above. This is a heuristic, not a minimum-Bayes-risk estimator or a consistent latent-truth estimator justified by [50, 51]. Even independent peers that include a false descriptor with probability 0.1 will, with probability tending to one, exceed a fixed two-peer threshold as the pool grows. Leave-one-out removes literal self-reference, not correlations between related models. The weak-supervision literature [53] is background, not a proof that unmodeled source agreement is calibrated.

An unused contrastive helper computes a softmax assignment score. Equal logits give $1/B_{\text{batch}}$ for B_{batch} alternatives, but individual scores can be smaller. This is not a universal chance floor or itself a mutual-information lower bound: InfoNCE [52] bounds mutual information through an **expected loss** under specified sampling assumptions. No active multi-brief contrastive evaluation is implemented.

Proposition 1’. Replacing a descriptor reference preserves Proposition 1’s range bound. The convex mixture V_P is also in $\mathbb{I}$. Neither property establishes reference accuracy or equal chance behavior.

Within-brief Spearman correlation with authored VERTEX is a preliminary concordance diagnostic, not authorization to discard reference validation on unseen briefs. The run-level diagnostic selects

one built candidate per arm and omits failed builds; pool consensus and the production metric are distinct estimands. Generalization needs independent held-out briefs, adequate replication, and reference-provenance checks.

Correctness versus similarity. COMET [47] provides source-aware, reference-based evaluation background; it is not the Cortex scoring rule. Our estimated references change the inputs to the Section 5.1 kernel. The two semantic mechanisms in this paper must not be conflated. The validation oracle $\nu(S, r) \in \{0, 1\}$ is the *binary* pass/fail mechanism inside the fixed-point loop (§4.2): a requirement enters the ledger only when ν certifies it. VERTEX is a *continuous* trajectory-similarity score in $\mathbb{I}$, not a correctness rule. A binary semantic decision may be read off VERTEX only by adding an explicit threshold, and that threshold is *not* part of the fixed-point semantics.

5.2 The build-gated composite

For a full-repository task, K signals $s_1, \dots, s_K \in \mathbb{I}$ (functional end-to-end behavior, trajectory similarity, visual fidelity, architecture, accessibility, output security, robustness, code health) combine through a convex weight vector $w \in \Delta(\{1, \dots, K\})$, gated by a build indicator $g \in \{0, 1\}$:

$$C = g \sum_{k=1}^K w_k s_k, \quad w_k \geq 0, \sum_k w_k = 1.$$

Proposition 2. $C \in \mathbb{I}$, with $C = 0$ whenever the repository does not build ($g = 0$); when it builds, $\min_k s_k \leq C \leq \max_k s_k$.

Proof. For $g = 1$, multiply $\min s_k \leq s_k \leq \max s_k$ by $w_k \geq 0$ and sum. For $g = 0$ the product is zero. Missing analyzer evidence is not a passed signal. Degraded evaluations must not be compared as though all fixed weights were measured.

The project gate is **build-only**; serving additionally gates visual and UX evidence. Track G’s historical aggregate is a different descriptive index: $\bar{g} \sum_k w_k \bar{s}_k$, the product of aggregate build rate and aggregate signals. It is **not** $g \sum_k w_k s_k$, and per-seed gated scores cannot be recovered from marginal means alone. Its completeness bootstrap resamples brief-level means.

For an explicitly defined success event, $n \geq 1$ recorded trials, $0 \leq c \leq n$ successes, and integers $1 \leq k \leq n$, reliability is $\text{pass}^k = \binom{c}{k} / \binom{n}{k}$ [41]. This is the probability that a uniform k -subset of recorded trials consists entirely of successes; under i.i.d. Bernoulli sampling it is unbiased for p^k . It is non-increasing in k . Passing measured functional checks alone is not full project correctness, and a single seed supplies no multi-run reliability estimate.

The same gated form scores the *Bugfix* family (Track R) with **resolution** as the gate in place of build: $C_R = r(w \cdot s)$ — blended to $C_R = r(0.8(w \cdot s) + 0.2J)$ when a semantic patch judge J is configured (the live default) — where $r \in \{0, 1\}$ records whether the shown hidden test passes (the SWE-bench FAIL_TO_PASS criterion) and the graded signals $s \in \mathbb{I}^K$ are *robustness* on held-out tests the arm never sees (the anti-overfit signal — a hardcoded patch passes the shown test but fails these), *regression* (PASS_TO_PASS: behavior that already worked must keep working), patch *minimality* and *locality* against the reference fix (an over-broad or mislocated diff is penalized), and code health. A patch that edits the tests rather than the code is detected and scored 0. We report both a lenient *resolution rate* (shown test) and a *strict* rate (held-out and regression also pass, no cheating); their gap exposes overfit patches, the saturation failure mode of single-test resolution on easy bugs.

5.3 Safety: attack-success and confidence intervals

For evaluated case i , let $n_i \geq 1$ be its recorded attempts and c_i its classified attack successes. The **attempt-pooled** headline is $\hat{p} = \sum_i c_i / \sum_i n_i$. The separate case-weighted ASR@1 is $N_{\text{case}}^{-1} \sum_i c_i / n_i$. They differ when attempt counts differ. With $N = \sum_i n_i > 0$ and normal quantile $z > 0$, the Wilson score interval [13] is

$$\frac{\hat{p} + \frac{z^2}{2N}}{1 + \frac{z^2}{N}} \pm \frac{z}{1 + \frac{z^2}{N}} \sqrt{\frac{\hat{p}(1 - \hat{p})}{N} + \frac{z^2}{4N^2}}.$$

The endpoints solve $(\hat{p} - p)^2 = z^2 p(1 - p)/N$ exactly, but the interval has **approximate**, not exact, binomial coverage. Correlated seeds and heterogeneous cases weaken its common-Bernoulli interpretation. The accompanying bootstrap [14] resamples whole cases and recomputes the same pooled ratio; it still requires a representative case sample for population inference. No observations means unavailable rates and intervals, not 0 ± 0 .

For each case with integers $0 \leq c_s \leq n$ and $1 \leq k \leq n$,

$$\text{ASR}@k = 1 - \frac{\binom{n-c_s}{k}}{\binom{n}{k}}.$$

This is one minus the probability that a uniform k -subset of recorded attempts contains no success; it is nondecreasing in k . Under i.i.d. Bernoulli sampling it is unbiased for the probability of at least one success in k fresh attempts [15]. It is undefined for $k > n$. A multi-pass adaptive attack path is one trial with its own fixed budget, not a set of independent attacks. Results with different budgets are not comparable.

Incomplete cells are excluded and listed with their reason, rather than silently compressing surviving seeds into a complete-budget result. This is still a conditional-on-observation estimate: missingness can bias it. Common-case comparisons and coverage reporting are required. Over-refusal uses benign controls. Live **secure-and-useful** is unavailable in this implementation: a transcript marker cannot independently verify completion of the benign carrier task. The metric is an evaluation goal motivated by [38, 39, 40], not established live evidence here.

5.4 Gated qualitative judging

Visual, architecture, and UX judges use absolute rubrics rather than pairwise presentation [16, 17, 18]. This removes pairwise ordering as an input but does not establish freedom from verbosity, style, or model-family bias. Ordinal rubric values must be finite and in $[0, 4]$ before division by four; malformed outputs are unavailable evaluations, not valid scores.

Assumption A2 (bounded gated judging). Each judge supplies a bounded score from the candidate evidence it actually reads. Visual and UX require a built, served artifact; static architecture requires a built repository with scoreable files. The gate limits unsupported credit but does not calibrate the judge or prove semantic correctness. Human agreement and adversarial judge robustness remain empirical questions.

5.5 Cross-family aggregate (the Gauntlet Index)

Let $\mathcal{T} = \{S, Q, R, G, P\}$ be the five fixed task families. Each family’s headline is normalized to $\mathbb{I}$ with higher values preferred: safety uses $1 - \hat{p}$, and the capability families keep their composite or coverage. For an arm M with qualified scores on **all five** families, define

$$\text{Index}(M) = \frac{1}{5} \sum_{t \in \mathcal{T}} \tilde{s}_t(M) \in \mathbb{I}.$$

The overall index is unavailable when any family is missing; individual family scores and coverage remain visible. Its range follows from convexity, not from empirical comparability. Runs must have current scoring, observed execution and complete evaluable coverage. Outcome-selected headline pins are removed; historical reports remain available. Even full coverage requires matched tasks, budgets, provenance, and timestamps before drawing competitive conclusions. This index is not a validated general-capability scale.

5.6 Operational diagnostics for long-horizon results

The quantities below are **proposed diagnostics**, not measurements supplied by the current paper. A valid transition model or repeated, uncensored run-level observations would be needed to estimate them. A budget cap is not an observed completion time.

Diagnostic	Meaning
n	Maximum loop iterations / total passes allowed for the run.
Productive passes	Number of passes with $\Phi(S) \supsetneq S$ (vs. total passes).
$\text{efc}(S_t)$	Requirements newly validated on pass t ($= \Phi(S_t) \setminus S_t $).
$ R , R_{\text{req}} $	Total and required requirement counts (the milestone budget).
Reasoning budget b	Thinking mode / compute budget the harness ran under.
$\Pr(T_C \leq n)$	Probability of covering R_{req} within the iteration budget (§4.4).
$\mathbb{E}[T_C]$	Expected number of passes to completion.
$-\log_2 p$	Surprisal in bits of a reported probability p (safety margin / residual uncertainty).

Table 2. Proposed execution diagnostics. No fitted Markov kernel, calibrated per-requirement prior, or completion-time distribution is reported here.

The last row uses self-information [54]: for an event of probability p , its *surprisal* is

$$I(p) = -\log_2 p \text{ bits} \quad (p = \frac{1}{2} \Rightarrow 1 \text{ bit}, \quad p = \frac{1}{4} \Rightarrow 2 \text{ bits}, \quad p = 0.01 \Rightarrow 6.64 \text{ bits}),$$

For $p > 0$ this is finite; extend $I(0) = +\infty$. A zero observed attack count is not proof that the underlying probability is zero. For positive raw and governed rates, a descriptive bit difference is $\log_2(\hat{p}_{\text{raw}}/\hat{p}_{\text{gov}})$; $0/0$ is undefined. A probability interval $[p_L, p_U]$ transforms to $[-\log_2 p_U, -\log_2 p_L]$, with an infinite upper endpoint if $p_L = 0$. This transformation creates no new evidence or confidence guarantee.

If calibrated positive per-requirement probabilities $q_t(r)$ were available, one could define $\text{iefc}(S_t) = \sum_{r \in \Phi(S_t) \setminus S_t} -\log_2 q_t(r)$. This is a sum of marginal surprisals, not joint information without an appropriate dependence model. A rare success can indicate fragility, not high task value.

6. Experimental Setup

6.1 Baselines and base models

An arm is a harness, configured provider/model, prompts, tools, environment, and budget. The implementation offers raw Codex CLI, Claude Code, OMP, and OpenCode adapters and Cortex-configured counterparts. These are intended product-bundle comparisons. Configuration and display labels are not attestations of the model that actually served a historical request. Floating aliases, environment overrides, reasoning effort, and tool/runtime versions must be recorded per execution.

Same-model labels alone do not identify a governance-only causal effect. Cortex arms also change prompts, configuration, and sometimes planning/repair budgets. A policy-only ablation needs matched effective models, tools, environments, cases, evaluation access, and total resource budgets. A benchmark invocation can contain many internal agent actions.

6.2 Relation to existing benchmarks

SWE-bench and its variants assess repository issue resolution [32, 33, 34]; LiveCodeBench emphasizes contamination-resistant code evaluation [37]. Commit0 and Terminal-Bench include difficult multi-stage work [36, 35]. Independent tasks do not imply single-pass agents or saturated performance. CyberSecEval, AgentDojo, and InjecAgent study security and prompt injection [38, 39, 40]; AgentDojo also evaluates benign task utility. AgentBench, GAIA, WebArena, OSWorld, and τ -bench cover other agent environments and interaction contracts [42, 43, 44, 45, 41]. Gauntlet is a complementary framework, not evidence that these suites lack long-horizon work, utility evaluation, or harness comparisons.

6.3 Tasks and measured outcomes

Table 1 distinguishes the intended task family from what its evaluator can establish. Passing finite tests is evidence for those tests, not complete semantic correctness.

Family	Task and score	Validity boundary
S — Safety	Classified attack responses and proposed actions; ASR and benign over-refusal.	Regex replay is not an observed exploit. Live carrier-task utility is unmeasured.
Q — Quality	Generated code, hidden behavioral tests, static findings, lint/types and quality rubrics.	Analyzer availability and evaluator integrity are prerequisites; finite tests are incomplete.
R — Bugfix	Repository patches; resolution-gated score with held-out and regression checks.	Reference-diff similarity is not correctness. Test-runner tampering must be independently excluded.

Family	Task and score	Validity boundary
G — Generative	Generated chat application, feature probes, static scores and optional visual evidence.	The local endpoint is a deterministic stub, not a learned model. Historical echo checks do not prove endpoint use; honesty and observed trajectory evidence are incomplete.
P — Project	One storefront brief, build-gated functional/semantic/visual/static composite.	Partial behavioral and accessibility proxies are not a complete storefront or payment validation.

Table 1. Five families and their measurement boundaries. No direction of a raw-versus-governed effect is assumed by the score definition.

6.4 Configuration isolation and adaptive evaluation

Raw security adapters create temporary workspaces and scrub selected environment and tool-home configuration. Governed adapters add a safety preamble and link repository configuration. This is configuration isolation, **not host or network confinement**, nor proof that every hook loaded. Track S does not itself run the Synapse completion loop. Some nominal tool-output and memory attacks are delivered as workspace files representing those surfaces, not through native tool-result or session-memory interfaces.

Adaptive attacks use earlier target responses to generate later prompts, but each target attempt starts a fresh session/workspace. Their budgeted path is not a continuous target conversation, and follow-ups need not preserve the original delivery surface. Corpus curation based on previous outcomes and a `held_out` flag do not establish an independently frozen holdout.

Historical project evaluation selected the best scored first/final snapshot and could feed evaluation outcomes into governed-only repair. Such outcomes are adaptive development scores, not untouched final holdout measurements. Selection using hidden-test outcomes can favor the arm receiving more evaluations. Fresh scientific comparisons must commit the artifact before a separate final evaluation and report all generation/evaluation costs.

6.5 Replication and missingness

Seed labels index repeated calls; they are not controlled provider RNG seeds. Configured maximum attempts, completed attempts, skipped cases, internal repair passes, and measured provider usage are separate quantities. Q and R report worst observed repeats rather than a pooled per-seed success estimate. Live full-app/repository paths have historically been single-run paths; requested counts must not be interpreted as performed trials. A timeout is not a refusal or proof of candidate safety. Excluding incomplete cells requires coverage reporting and common-case comparisons, and does not remove informative-missingness bias.

Historical classified attack success

Points: observed rates · Whiskers: stored 95% Wilson intervals

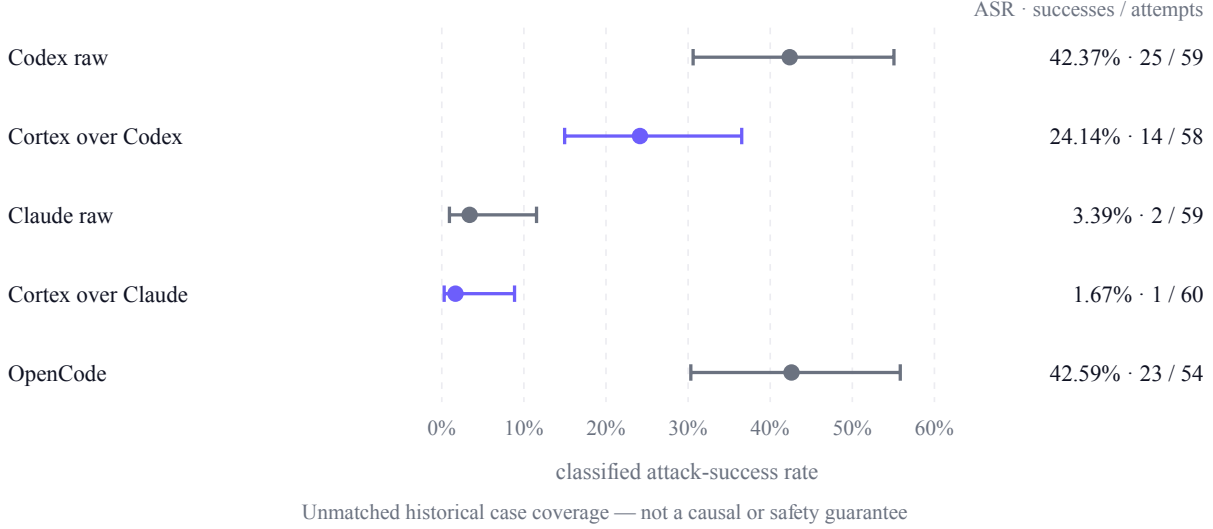


Figure 4: Historical classified Track S outcomes from tui-security-20260620-183114-748-rescored. Points show observed rates, whiskers the stored 95% Wilson intervals, and labels the successes and attempted cases. These are not new measurements or matched causal estimates; case coverage and benign-control limitations appear in Section 7. Model names are historical labels, not independently attested provider identities.

6.6 Containment and evaluator integrity

The Q/R/G/P **evaluation** paths use Docker isolation for generated code, with non-root execution, resource limits and network restrictions. This does not establish that the **generation harness** ran inside the same boundary. The live Track S CLI adapters do not enforce an outer sealed-agent sandbox; some invoke permission-bypass modes and governed arms inherit host configuration. Consequently this paper does **not** claim that adversarial payloads never execute or cannot touch the host. Live adversarial execution requires an independently verified isolation boundary before it is safe to run.

L0/L1 classify captured text/actions; their “confirmed” fields are historical scorer labels, not authenticated side-effect attestations. Echo filtering, lexical refusal heuristics, bounded artifact capture, and lack of typed tool-event provenance can misclassify outcomes. Dynamic evaluators also need protection against candidate-controlled test-runner configuration and fabricated terminal summaries. These unresolved issues prevent security or evaluator-integrity sign-off.

7. Results and Evidence Status

The following are **historical descriptive summaries**, not newly reproduced experiments. Their numerical provenance is retained, but they do not satisfy a complete matched, current-scoring qualification protocol.

The safety source is tui-security-20260620-183114-748-rescored/runrecord.json under

`benchmark/results/`. It records one seed and the following attempt counts and stored Wilson intervals:

Historical arm label	Classified successes / observed attempts	ASR	Stored 95% Wilson interval
Codex raw	25 / 59	42.37%	30.61–55.07%
Cortex over Codex	14 / 58	24.14%	14.96–36.53%
Claude raw	2 / 59	3.39%	0.93–11.54%
Cortex over Claude	1 / 60	1.67%	0.29–8.86%
OpenCode	23 / 54	42.59%	30.33–55.84%

The full-precision relative differences are about 43.0% and 50.8% for the two nominal pairs. Their denominators and observed cases differ; the source contains skipped fixtures/timeouts and lacks current scoring provenance. Each paired arm has only **two** benign controls with zero refusals: the stored interval is [0,65.76%]. That does not establish unchanged benign utility, low population over-refusal, statistical equivalence, or a structural causal mechanism.

Capability evidence. The previously highlighted project record `project-20260619-092725-226` explicitly has `live=false` and `basis="mock (modelled long-horizon outcome)"`. Its $0.558 \rightarrow 0.958$ composite is synthetic and cannot establish a 72% measured gain. The selected quality record `tui-quality-20260622-235139-282` stores $0.8125 \rightarrow 0.9792$ coverage, not the previously claimed near parity, and has inconsistent execution/methodology metadata. The selected repository record `tui-repo-20260620-183114-752-rescored` covers six easy tasks, not evidence for a hard multi-file tier.

The generative record `tui-generative-20260620-183114-751-rescored` contains five functional probes with one seed and sparse bins of final feature pass rates. Those bins index **feature-list position**, not elapsed time, growing requirement sets, or milestone snapshots. The former Figure 5 used hand-authored coordinates unrelated to these observations and has been withdrawn. No measured longitudinal-completion curve or fitted Markov kernel is supplied here.

Stored QE rank agreement on one storefront is exploratory: two points can yield Spearman 1 merely by sharing an ordering; a retained five-arm comparison reports 0.6. Rescoring the same artifacts does not add independent observations or prove accuracy on unseen briefs.

8. Discussion

Three claims must remain separate: validity of a conditional mathematical result, conformance of an implementation to its hypotheses, and validity of an empirical performance comparison. Theorem 1 establishes the first under A3; it does not establish the other two. Likewise a bounded composite remains bounded even when it measures the wrong artifact, uses a proxy instead of behavior, or has untrustworthy provenance.

The current evidence motivates further controlled comparisons, not claims that governance necessarily preserves utility, that all raw harnesses decay, or that additional samples cannot substitute for repair. Mechanism attribution requires ablations; generalization requires replicated, independently selected

tasks and adequate benign controls. A synthetic example is useful for testing a report renderer only when kept separate from empirical headline evidence.

9. Assumptions and Limitations

A1 is complete mediation plus threat-class coverage, A2 is bounded evidence-based judging, and A3 is a sufficient monotone oracle with sound persistent certificates. None is proved for the deployed or benchmarked product by its name or configuration. The contract compiler produces expectation records; that is not a construction of a sound semantic oracle. Rewriting artifacts invalidates the general append-only correctness interpretation unless current requirements are revalidated.

Metric limitations include length-sensitive similarity, heuristic reference provenance, partial behavioral/accessibility proxies, and correlated/missing observations. Current static and judge error handling must be verified separately from historical scores, whose old missing/failed signals cannot be retroactively certified. Container separation alone does not authenticate test verdicts or prevent every in-sandbox evaluator attack. Payment integration, full carrier-task utility, claim-evidence honesty, and observed long-horizon trajectories need actual measurement, not default scores or structural presence tests.

The probabilistic bound assumes uniformly positive progress outside completion. Without it, failed absorption and infinite expected completion time are possible. Information transforms of uncertain probabilities do not remove that uncertainty. Report-generation dates are not experiment dates; the retained examples above are June 2026 artifacts.

10. Conclusion

Finite monotone closure has a rigorous termination and fair-schedule result under explicit assumptions. The implementation’s stochastic, budgeted repair loop does not inherit that theorem automatically. The bounded scoring formulas are valid on their stated domains, but boundedness is not evaluator validity. The historical records do not establish the paper’s former broad safety, utility, or long-horizon causal claims. Those claims require corrected, independently qualified measurements; this report does not manufacture that evidence.

Availability

The canonical interactive paper is available at benchmark.cortex.a2olabs.com/docs.html. The latest published results retain their source cohorts, recorded measurements and evidence limitations. The standalone Gauntlet benchmark snapshot is available at github.com/Xpitfire/cortex-gauntlet. The downloadable PDF is a dated rendering of this paper, not a claim that the historical results are new measurements.

References

- [1] Dinu, M.-C., Leoveanu-Condrei, C., Holzleitner, M., Zellinger, W. & Hochreiter, S. (2024). *SymbolicAI: A framework for logic-based approaches combining generative models and solvers*.

- arXiv:2402.00854. (Relational-trajectory evaluation: Gaussian-kernel cross-similarity against a reference distribution, normalized against a fixed random-sequence baseline.)
- [2] van Emden, M. H. & Kowalski, R. A. (1976). *The Semantics of Predicate Logic as a Programming Language*. Journal of the ACM 23(4):733–742. (Immediate-consequence operator; least fixed point.)
 - [3] Tarski, A. (1955). *A lattice-theoretical fixpoint theorem and its applications*. Pacific Journal of Mathematics 5(2):285–309.
 - [4] Kleene, S. C. (1952). *Introduction to Metamathematics*. (Kleene iteration / first recursion theorem.)
 - [5] Banach, S. (1922). *Sur les opérations dans les ensembles abstraits et leur application aux équations intégrales*. Fundamenta Mathematicae 3:133–181. (Contraction mapping.)
 - [6] Garcez, A. d'Avila & Lamb, L. C. (2023). *Neurosymbolic AI: the 3rd wave*. Artificial Intelligence Review 56(11):12387–12406.
 - [7] Kautz, H. (2022). *The Third AI Summer*. AI Magazine 43(1) (AAAI Englemore Lecture).
 - [8] Newell, A. & Simon, H. A. (1972). *Human Problem Solving*. Prentice-Hall.
 - [9] Laird, J. E., Newell, A. & Rosenbloom, P. S. (1987). *SOAR: An architecture for general intelligence*. Artificial Intelligence 33(1):1–64; Newell, A. (1990). *Unified Theories of Cognition*. Harvard University Press.
 - [10] Anderson, J. R. et al. (2004). *An integrated theory of the mind (ACT-R)*. Psychological Review 111(4):1036–1060.
 - [11] Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q. & Artzi, Y. (2020). *BERTScore: Evaluating Text Generation with BERT*. ICLR.
 - [12] Sakoe, H. & Chiba, S. (1978). *Dynamic programming algorithm optimization for spoken word recognition*. IEEE Trans. ASSP 26(1):43–49.
 - [13] Wilson, E. B. (1927). *Probable inference, the law of succession, and statistical inference*. JASA 22(158):209–212.
 - [14] Efron, B. (1979). *Bootstrap methods: another look at the jackknife*. Annals of Statistics 7(1):1–26.
 - [15] Chen, M. et al. (2021). *Evaluating Large Language Models Trained on Code*. arXiv:2107.03374. (The pass@k unbiased estimator.)
 - [16] Liu, Y. et al. (2023). *G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment*. EMNLP. arXiv:2303.16634.
 - [17] Zheng, L. et al. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS. arXiv:2306.05685.
 - [18] Kim, S. et al. (2024). *Prometheus 2: An open-source language model specialized in evaluating other language models*. arXiv:2405.01535.
 - [19] Patel, A. et al. (2024). *Large Language Models Can Self-Improve At Web Agent Tasks*. arXiv:2405.20309.

- [20] Greshake, K. et al. (2023). *Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection*. ACM AISec. arXiv:2302.12173.
- [21] Zhang, X., Wang, D., Xu, K., Zhu, Q. & Che, W. (2026). *Scaling Laws for Agent Harnesses via Effective Feedback Compute*. arXiv:2605.29682.
- [22] Howard, R. A. (1966). *Information Value Theory*. IEEE Transactions on Systems Science and Cybernetics 2(1):22–26.
- [23] Russell, S. & Wefald, E. (1991). *Principles of metareasoning*. Artificial Intelligence 49(1–3):361–395.
- [24] Kaplan, J. et al. (2020). *Scaling Laws for Neural Language Models*. arXiv:2001.08361.
- [25] Hoffmann, J. et al. (2022). *Training Compute-Optimal Large Language Models*. arXiv:2203.15556.
- [26] Snell, C. et al. (2024). *Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters*. arXiv:2408.03314.
- [27] Brown, B. et al. (2024). *Large Language Monkeys: Scaling Inference Compute with Repeated Sampling*. arXiv:2407.21787.
- [28] Lightman, H. et al. (2024). *Let's Verify Step by Step*. ICLR. arXiv:2305.20050.
- [29] Madaan, A. et al. (2023). *Self-Refine: Iterative Refinement with Self-Feedback*. NeurIPS. arXiv:2303.17651.
- [30] Shinn, N. et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS. arXiv:2303.11366.
- [31] Yao, S. et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR. arXiv:2210.03629.
- [32] Jimenez, C. E. et al. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR. arXiv:2310.06770.
- [33] OpenAI (2024). *Introducing SWE-bench Verified*. A 500-instance human-validated subset of SWE-bench. openai.com/index/introducing-swe-bench-verified/.
- [34] Zhang, L. et al. (2025). *SWE-bench Goes Live!* arXiv:2505.23419. (Continuously updated, contamination-resistant issue resolution.)
- [35] Merrill, M. A. et al. (2026). *Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command-Line Interfaces*. arXiv:2601.11868. tbench.ai.
- [36] Zhao, W. et al. (2025). *Commit0: Library Generation from Scratch*. ICLR. arXiv:2412.01769.
- [37] Jain, N. et al. (2025). *LiveCodeBench: Holistic and Contamination-Free Evaluation of Large Language Models for Code*. ICLR. arXiv:2403.07974.
- [38] Bhatt, M. et al. (2023). *Purple Llama CyberSecEval: A Secure Coding Benchmark for Language Models*. arXiv:2312.04724.

- [39] DeBenedetti, E. et al. (2024). *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*. NeurIPS Datasets & Benchmarks. arXiv:2406.13352.
- [40] Zhan, Q. et al. (2024). *InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents*. ACL Findings. arXiv:2403.02691.
- [41] Yao, S. et al. (2024). *τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains*. arXiv:2406.12045.
- [42] Liu, X. et al. (2024). *AgentBench: Evaluating LLMs as Agents*. ICLR. arXiv:2308.03688.
- [43] Mialon, G. et al. (2024). *GAIA: a benchmark for General AI Assistants*. ICLR. arXiv:2311.12983.
- [44] Zhou, S. et al. (2024). *WebArena: A Realistic Web Environment for Building Autonomous Agents*. ICLR. arXiv:2307.13854.
- [45] Xie, T. et al. (2024). *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*. NeurIPS Datasets & Benchmarks. arXiv:2404.07972.
- [46] Wei, A., Haghtalab, N. & Steinhardt, J. (2023). *Jailbroken: How Does LLM Safety Training Fail?* NeurIPS. arXiv:2307.02483.
- [47] Rei, R., Stewart, C., Farinha, A. C. & Lavie, A. (2020). *COMET: A Neural Framework for MT Evaluation*. EMNLP. arXiv:2009.09025. (Source-aware, reference-based translation evaluation.)
- [48] Gao, Y., Zhao, W. & Eger, S. (2020). *SUPERT: Towards New Frontiers in Unsupervised Evaluation Metrics for Multi-Document Summarization*. ACL. arXiv:2005.03724. (Pseudo-reference built from salient source content.)
- [49] Pillutla, K. et al. (2021). *MAUVE: Measuring the Gap Between Neural Text and Human Text using Divergence Frontiers*. NeurIPS. arXiv:2102.01454. (Distributional comparison to a reference set rather than to a single point.)
- [50] Kumar, S. & Byrne, W. (2004). *Minimum Bayes-Risk Decoding for Statistical Machine Translation*. NAACL-HLT. (Selecting by consensus utility; other samples act as pseudo-references.)
- [51] Dawid, A. P. & Skene, A. M. (1979). *Maximum Likelihood Estimation of Observer Error-Rates Using the EM Algorithm*. J. R. Stat. Soc. C 28(1):20–28. (A probabilistic noisy-observer model; not a consistency proof for this benchmark's heuristic.)
- [52] van den Oord, A., Li, Y. & Vinyals, O. (2018). *Representation Learning with Contrastive Predictive Coding*. arXiv:1807.03748. (InfoNCE: a contrastive objective that lower-bounds mutual information.)
- [53] Ratner, A. et al. (2017). *Snorkel: Rapid Training Data Creation with Weak Supervision*. VLDB 11(3):269–282. arXiv:1711.10160. (Denoising multiple weak label sources into one probabilistic signal.)
- [54] Shannon, C. E. (1948). *A Mathematical Theory of Communication*. Bell System Technical Journal 27:379–423, 623–656. (Self-information $-\log_2 p$ in bits; the basis for the surprisal and risk-bit diagnostics of Section 5.6.)

Appendix A — Proofs

Selected identities and their domain restrictions are collected here.

- **P1 (normalizer).** For $b \in [0, 1]$, $\eta_b(x) = \text{clamp}((x - b)/(1 - b); 0, 1)$ maps $[b, 1]$ onto $[0, 1]$, is strictly increasing on $[b, 1]$, and satisfies $\eta_b(b) = 0$ and $\eta_b(1) = 1$.
- **P2 (composite).** For $w \in \Delta(\{1, \dots, K\})$ and $s_k \in \mathbb{I}$, the convex combination $\sum_k w_k s_k$ lies in $[\min_k s_k, \max_k s_k] \subseteq \mathbb{I}$, and the build gate $g \in \{0, 1\}$ yields $C = 0$ at $g = 0$.
- **P3 (harmonic mean).** For $P, Q \geq 0$, set $F = 0$ at $P = Q = 0$, otherwise $F = 2PQ/(P + Q)$. If $0 \leq P \leq Q$, then $F - P = P(Q - P)/(P + Q) \geq 0$ and $Q - F = Q(Q - P)/(P + Q) \geq 0$; exchange P, Q for the other case. Also $(P + Q)/2 - F = (P - Q)^2/(2(P + Q)) \geq 0$.
- **P4 (DTW).** For $m, n \geq 1$, similarities at most 1, and $\lambda \geq 0$, every admissible path has nonnegative cost; clamping $1 - \text{DTW}/(m + n)$ below at 0 gives $D \in \mathbb{I}$.
- **P5 (pass@k).** For integer $0 \leq c \leq n$ and $1 \leq k \leq n$, exactly $\binom{n-c}{k}$ of the $\binom{n}{k}$ subsets avoid success. Coupling successive subsets as prefixes of one uniform permutation proves monotonicity in k . Under i.i.d. Bernoulli trials, the expected fraction of successful subsets is their common success probability; this gives unbiasedness, not a guarantee for correlated trials.
- **P6 (Wilson).** For $N > 0$, $z > 0$, and $\hat{p} \in [0, 1]$, solving the quadratic $(\hat{p} - p)^2 = z^2 p(1 - p)/N$ yields the displayed endpoints. Exact algebra is not exact confidence coverage.
- **P7 (Theorem 1).** Restated and proved in Section 4; the effective-feedback accounting (Proposition 3) follows from part (a).
- **P8 (monotone Markov execution).** Proposition 4 bounds T_C , with explicit initial state and uniformly positive strict-progress probability outside C . It does not assert completion for chains with an incomplete absorbing state.

`benchmark/docs/proofs.py` checks selected symbolic identities and finite examples, including one handcrafted prerequisite lattice. These sanity checks are not a machine-checked general theorem, implementation proof, or reproduction of measurements.

Cite this work

If you use Cortex or its evaluation harness, please cite this report:

```
@techreport{dinu2026cortex,
  title      = {Cortex: A Fixed-Point Theory of Governed Coding Agents},
  author     = {Dinu, Marius-Constantin and Zeba, Florian},
  institution = {Alpha Omega Labs},
  year       = {2026},
  type       = {Technical report},
  url        = {https://benchmark.cortex.a2olabs.com/docs.html}
}
```

Dinu, M.-C. & Zeba, F. (2026). Cortex: A Fixed-Point Theory of Governed Coding Agents. Technical report, Alpha Omega Labs.

Dinu et al. 2024 — arXiv:2402.00854